B

FraudEngine

GOEMAN GLOBAL FINANCE · CONFIDENTIAL — FOUNDER · **JUNE 29, 2026**

Strategic guidance only — not legal, financial, tax, or investment advice. Securities, money-transmission, and corporate-formation decisions must be reviewed by licensed counsel and a CPA before you act. Sections flagged "⚖ see counsel" are where this matters most.

Contents

Technology Strategy

> **Status: v2 / production north-star — not current-phase scope.** This describes the target real-time fraud platform (Kafka + Flink + Transformer serving + feature store + model registry + lakehouse), in the same category as Temporal/Conductor and the Go reimplementation. The prototype does **not** implement this platform. What *is* built is **Stage 1**: an in-process fraud seam (`backend/src/services/fraudService.ts` + the append-only `fraud_decisions` table) that screens the money path and shapes its event/score/decision/audit contract to grow into the design below. See `FraudEngine-GapAnalysis.md` for the conformance table, what Stage 1 closes, and the phased roadmap (Stages 2–4 remain a locked-architecture decision).

In this evolved architecture, the entire fintech/bank becomes a pure event-driven platform where *every* product (payments, lending, accounts, cards, investments, support, etc.) streams *all* its data in real time to a unified Kafka backbone. This creates a single source of truth for the full user lifecycle.

The fraud team gets full autonomy as a "listener-first" consumer: they subscribe to the stream(s), maintain their own processing jobs, and dynamically decide which Transformer model (or ensemble) to invoke for inference *or* trigger learning/retraining pipelines — for testing (shadow/canary) or production — without touching core product code. This decouples fraud completely while keeping sub-50–100ms end-to-end latency for decisions.

This builds directly on the previous data layer/orchestrator (Flink + feature store + Backbase-style orchestration) but makes the stream the universal bus and gives fraud team self-service control via model registry + config-driven routing.

High-Level Architecture Overview

- **Producers (All Products) → Unified Event Stream (Kafka) → Fraud Listening & Processing Layer (Flink jobs owned/controlled by fraud team) → Dynamic Transformer Model Router + Serving → Decisions back to stream + actions** (block, challenge, flag, etc.).
- Everything is immutable events (standardized Avro/Protobuf schema with user_id, timestamp, context, payload). Schema Registry enforces consistency.
- Fraud team operates like an internal "fraud intelligence platform" on top of the shared stream — no direct database access needed; everything is event-driven.

Data Flow (per transaction/journey event):

1. Any product emits an event (e.g., "payment_initiated", "login_attempt", "account_viewed", "KYC_step_completed").

2. Kafka ingests at massive scale (millions of events/sec).
3. Fraud Flink job(s) consume (dedicated consumer group, partitioned by user_id for stateful processing).
4. Flink enriches in real time (feature store lookup for user profile, session state, historical sequences).
5. Fraud logic/config decides: "Route this event to Transformer Model vX.Y (prod) or vX.Y-test (shadow)".
6. Async inference call to the chosen model endpoint → risk score + explanation.
7. Decision emitted to output Kafka topic (consumed by orchestrator/products for real-time action + UI adaptation).
8. Outcome (actual fraud or not) loops back as a later event for labeling/learning.

Testing vs Production Models:

- **Shadow mode**: Fraud team spins up parallel Flink job or branch in the main job that runs the test Transformer alongside prod (logs scores to a separate topic for offline comparison — zero impact on live decisions).
- **Canary/A/B**: Config routes 1–5% of traffic (by user hash or risk tier) to test model; monitor metrics in real time.
- **Promotion**: Fraud team updates a simple config (in etcd, Consul, or Databricks) → prod job instantly switches routing. Rollback in seconds.
- **Learning/Retraining**: Stream feeds a lakehouse (Databricks Delta Lake or Iceberg). Fraud team triggers training pipelines (e.g., via Airflow/Databricks Workflows) on new labeled data. New Transformer version registered → tested in shadow → promoted.

This gives the fraud team "listen + decide + invoke" superpowers with full auditability (every model call logged with version + input/output).

Core Components & Tech (2026 Best-Fit for Low Latency + Transformer Support)

All components are cloud-native, Kubernetes-orchestrated, and compliant (GDPR, SOC2, explainability via SHAP/LIME for Transformers).

Component	Recommended Tech (2026)	Why It Fits Fraud Team Autonomy + Transformers + Low Latency
Event Streaming Backbone	Apache Kafka (Confluent Cloud or Redpanda) + Schema Registry	Universal bus for *all* products. Exactly-once, replayable for testing. Fraud subscribes independently.
Fraud Listening/Processing	Apache Flink (2.2+ with ML_PREDICT support) — fraud-owned jobs	Stateful per-user sequences (perfect for Transformer input). AsyncDataStream for non-blocking model calls. Fraud team deploys/maintains their own jobs via self-service GitOps.
Real-Time Features/Context	Feature Store (Tecton/Databricks online store on Redis/Cassandra)	Sub-10ms lookups for user journey state. Stream updates features live.
Dynamic Model Router	Lightweight Flink ProcessFunction + config service (or simple Drools rules) controlled by fraud team	Fraud team updates "which Transformer for which event/risk" in real time via API/UI. No code change needed.
Transformer Model Serving	Triton Inference Server (NVIDIA) or vLLM/TensorRT-LLM on GPU pods; or managed (SageMaker, Vertex AI, Databricks Model Serving)	Optimized for Transformers (quantization, batching, time-aware embeddings like FraudTransformer-style). <20–50ms inference even on sequences. Supports multiple versions side-by-side.
Model Registry & Experiments	MLflow (or Databricks Model Registry) + experiment tracking	Fraud team trains (offline on lakehouse data from stream), registers versions, compares test/prod metrics. Shadow testing built-in.
Training/Learning Data Lake	Databricks Lakehouse (or Snowflake) fed by Kafka → Flink → Delta Lake	Stream continuously materializes training datasets + labels. Fraud triggers online learning or full retrains.
Orchestrator Integration	Backbase/Temporal hooks consume fraud decisions from output topic	Full journey adaptation (e.g., "block + show challenge UI" based on Transformer score).

Transformer-Specific Notes:

- Use sequence models (TabTransformer, Time-Aware GPT variants, or custom encoder-only Transformers) trained on user journey sequences (clicks + transactions).
- Input: Real-time windowed sequence from Flink state + features.
- Low-latency tricks: Quantized models (8-bit/4-bit), distillation, GPU acceleration, async calls in Flink. End-to-end <100ms is standard in production fraud systems today.

How the Fraud Team Operates Day-to-Day (Minimal Friction)

- **Listen**: Deploy a Flink job (or use a managed platform like Databricks Streaming) that auto-subscribes to relevant topics. Filter/transform as needed.
- **Decide Model**: Via a fraud-controlled config dashboard: "For high-risk payments → use Transformer-v2.3-prod; for testing → shadow with v2.4-experimental".
- **Invoke**: Flink routes the enriched event payload to the chosen endpoint (REST/gRPC). Supports ensembles (multiple Transformers + rules).
- **Learn/Iterate**: Monitor drift via stream metrics → trigger retrain on lakehouse data → register new model → shadow test → promote.
- **Testing Safety**: Historical replay (Kafka + Flink savepoints) or shadow mode on live traffic. Metrics dashboards compare models without production risk.
- **Autonomy Without Chaos**: Fraud jobs run in isolated namespaces/consumer groups. Platform team handles Kafka/Flink infra; fraud owns logic/models.

Latency & Scale: Flink + async inference + optimized serving = sub-second decisions at millions of TPS. Transformers add negligible overhead with modern serving (proven in 2025–2026 fintech deployments).

Compliance & Observability:

- Every decision includes model version, confidence, SHAP explanation → audit topic.
- Bias/drift monitoring built into the registry.
- Consent & PII: Stream redaction + on-demand purge.

This architecture turns fraud from a siloed "batch model" team into a real-time intelligence layer that evolves continuously with the business. It fits perfectly into the broader user journey orchestrator (same stream, same Flink cluster, shared feature store) while giving fraud complete ownership.

If you share your current Kafka/Flink maturity, preferred cloud, or specific Transformer use case (e.g., sequence vs tabular), I can provide sample Flink job pseudocode, model registry config examples, or a more detailed diagram description. This is production-proven at scale in banks today.